

Labo 05 — Réponses commentées

Question 1 — 950 Mo refusés par la sécurité

Réponse. Trois arguments hors espace disque :

1. **Le code source est distribué avec l'image.** Toute personne ayant accès au registry — ou toute image ayant fuité — dispose des sources de l'application, de sa configuration de build, parfois de commentaires ou d'URL internes.
2. **La surface d'attaque explose.** L'image embarque un compilateur, Maven, un shell, `curl`, `git`. Un attaquant qui obtient l'exécution de code dans le conteneur dispose immédiatement de quoi télécharger, compiler et exécuter un outil.
3. **Le rapport de vulnérabilités devient inexploitable.** Une image de 950 Mo produit couramment plusieurs centaines de CVE, dont l'immense majorité concerne des composants jamais utilisés en production. Personne ne trie 300 lignes : les vraies alertes se noient.

Le plus difficile à corriger autrement : le **premier**. On peut à la rigueur désinstaller des outils, mais les sources ont été copiées dans une couche — les supprimer plus tard ne les retire pas de l'image (labo 02). Seul un stage jeté les fait réellement disparaître.

Nuance. Le poids n'est pas qu'une question de disque : il se paie en bande passante à chaque déploiement, sur chaque nœud, et en temps de démarrage lors d'un incident.

Exemple.

```
docker history mon-api:1.0 --format 'table {{.Size}}\t{{.CreatedBy}}' | head -6
docker run --rm mon-api:1.0 sh -c 'ls /app/src && which mvn javac git curl'
```

Question 2 — Le rôle de chaque stage

Réponse. Le **dernier** `FROM` produit l'image finale. Les précédents sont des environnements de construction intermédiaires, jetés à la fin : rien de ce qu'ils contiennent n'entre dans l'image, sauf ce qu'un `COPY --from` en extrait explicitement. Si aucun `COPY --from` ne référence le deuxième stage et qu'il n'est pas la base d'un autre, **BuildKit ne le construit pas du tout**.

Pourquoi. BuildKit ne raisonne pas ligne à ligne mais construit un graphe de dépendances entre les stages, puis n'exécute que ce dont le résultat final dépend. C'est la « construction paresseuse ».

Nuance. Ce comportement est très pratique — on peut garder un stage `test` ou `debug` dans le Dockerfile sans ralentir le build de production — mais il surprend : un stage qui « ne s'exécute pas » n'est pas un bogue. Pour le forcer, on le cible explicitement : `docker build --target test .`

Exemple.

```
docker build --target build -t api:etape-build . # s'arrêter au stage nommé "build"
docker run --rm -it api:etape-build sh           # inspecter ce qu'il contient
```

Question 3 — `COPY --from=build /app /app`

Réponse. Le `COPY --from` recopie **tout** le dossier de travail du stage de build : les sources, `target/` en entier (classes, tests, rapports), et tout ce que Maven y a laissé. Correction :

```
COPY --from=build /app/target/api.jar /app/api.jar
```

Pourquoi. Le multi-stage ne filtre rien tout seul : il vous **donne la possibilité** de ne prendre que l'artefact. Si vous copiez tout, vous reproduisez le problème d'origine, en plus discret.

Nuance. C'est l'erreur la plus fréquente sur les Dockerfiles multi-stage « en apparence corrects ». Elle est d'autant plus insidieuse qu'elle est invisible dans la structure du fichier : le Dockerfile a deux stages, la revue de code passe, et l'image contient toujours les sources. Le réflexe à prendre : après chaque build, vérifier ce que l'image contient réellement.

Exemple.

```
docker run --rm mon-api:1.0 ls -la /app
# api.jar seul = correct ; src/, target/, pom.xml = le COPY est trop large
```

Question 4 — `ng serve` en production

Réponse. Quatre raisons :

1. **C'est un serveur de développement.** Non optimisé, mono-processus, sans compression ni cache HTTP, il n'est ni conçu ni testé pour encaisser du trafic réel — la documentation Angular le dit explicitement.
2. **Le build n'est pas de production.** `ng serve` sert du code non minifié, avec les *source maps* et sans `--configuration production` : le code source TypeScript est distribué aux navigateurs, et les performances sont sans rapport avec le résultat final.
3. **L'image contient Node et les sources.** Plusieurs centaines de Mo, un runtime JavaScript complet et le code du projet, là où il ne faudrait que des fichiers statiques.
4. **Le rechargement à chaud surveille le système de fichiers.** Consommation CPU permanente et comportements imprévisibles en conteneur.

À la place : `ng build --configuration production` dans un stage `node`, puis copie du seul dossier `dist/` dans une image `nginx:alpine`. L'image finale ne contient plus que des fichiers statiques et un serveur web éprouvé.

Nuance. « Ça marche en recette » est justement le problème : la recette valide alors un artefact qui n'est pas celui de la production. C'est la même faute que déployer un JAR compilé avec un profil de développement.

Exemple.

```
FROM node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/dist/mon-app/browser /usr/share/nginx/html
```

Question 5 — `UnsatisfiedLinkError` après le passage à Alpine

Réponse. Alpine utilise **musl** au lieu de la `glibc`. Une bibliothèque native embarquée par le générateur de PDF (rendu de polices, images, compression) a été compilée pour `glibc` : elle ne peut pas se charger sous musl, d'où l'`UnsatisfiedLinkError`.

Pourquoi. Le code Java pur est portable, mais tout ce qui passe par JNI dépend de binaires natifs liés à une implémentation précise de la bibliothèque C.

Nuance. Le vrai enseignement porte sur la **méthode**, pas sur Alpine. Le défaut n'est pas d'avoir choisi Alpine, mais d'avoir traité un changement d'image de base comme un changement cosmétique. Un changement de base est un changement d'environnement d'exécution : il se valide comme une montée de version — sur la totalité des parcours fonctionnels, y compris les traitements par lots, les exports et les tâches planifiées, qui sont rarement couverts par les tests d'intégration. Le symptôme apparaît d'ailleurs **deux semaines après**, sur une fonction peu utilisée : c'est exactement la signature de ce type de régression. Solutions : soit revenir à `-jammy`, soit installer `gcompat` sur Alpine (au prix d'une partie du gain, et sans garantie).

Exemple.

```
docker run --rm eclipse-temurin:21-jre-alpine sh -c 'ldd --version 2>&1 | head -1' # musl
docker run --rm eclipse-temurin:21-jre-jammy sh -c 'ldd --version 2>&1 | head -1' # GNU libc
```

Question 6 — Multi-stage et secrets

Réponse. Parce qu'un stage jeté ne produit **aucune couche** dans l'image finale : le fichier n'y est ni présent ni masqué, il n'existe simplement pas. Un `rm`, lui, ajoute une couche qui masque le fichier sans le supprimer des données de l'image.

Le multi-stage ne protège **pas** dans trois cas : si vous copiez le secret dans le stage final (`COPY --from=build /app /app` trop large), si vous l'exposez en `ENV` ou en `ARG` visible dans `docker history`, ou si vous le laissez dans le **cache de build**, que BuildKit peut exporter et partager avec la CI (`--cache-to`).

Pourquoi. Ce qui compte n'est pas ce qu'on voit à l'exécution, mais ce qui a été matérialisé dans une couche de l'image ou de son cache.

Nuance. La solution vraiment propre reste le *secret mount* de BuildKit (`RUN --mount=type=secret,id=...`), qui rend le fichier disponible le temps d'une instruction sans jamais l'écrire nulle part. Le multi-stage est une protection **structurelle** ; le secret mount est une protection **explicite**. En entreprise, on utilise les deux. Sujet du labo 08.

Exemple.

```
docker save mon-api:1.0 | tar -x -O 2>/dev/null | grep -c "MA_CLE_SECRETE" # doit valoir 0
docker history --no-trunc mon-api:1.0 | grep -i secret # rien attendu
```

Question 7 — JAR monolithique contre JAR en couches

Réponse.

Stratégie	Transféré au déploiement d'un correctif d'une ligne
(a) <code>COPY target/api.jar</code>	La couche entière, soit ~50 Mo
(b) <code>layertools</code>	La seule couche <code>application</code> , soit quelques centaines de Ko à 5 Mo

Pourquoi. Le transfert est différentiel **à la granularité de la couche**. Un JAR copié en bloc forme une couche unique : un octet modifié à l'intérieur change le digest de toute la couche. Découpé, seule la partie « code applicatif » change ; les dépendances gardent leur digest et ne sont pas retransférées.

Nuance. (a) reste acceptable dans beaucoup d'entreprises, pour trois raisons : le registry est souvent sur le même réseau que les serveurs (50 Mo se transfèrent en une seconde) ; le nombre de déploiements quotidiens est faible ; et la complexité ajoutée par `layertools` doit être maintenue. Le calcul change dès qu'on déploie plusieurs dizaines de fois par jour, sur de nombreux nœuds, ou à travers un lien lent — cas typique du edge ou du multi-région.

Exemple.

```
docker history mon-api:1.0 --format 'table {{.Size}}\t{{.CreatedBy}}' | head -6
# (b) montre plusieurs couches COPY distinctes au lieu d'une seule de 50MB
```

Question 8 — Le build qui passe de 90 s à 7 min

Réponse. Le nouvel agent a un **cache de build vide**. Toutes les étapes, y compris le téléchargement des dépendances Maven ou npm, sont rejouées depuis zéro. Le Dockerfile est correct : ce n'est pas lui, c'est l'environnement.

Pourquoi. Le cache de build est **local au daemon**. Il ne voyage pas avec le Dockerfile ni avec le dépôt Git. Un agent neuf, une machine recréée à chaque job, ou un `docker system prune` produisent le même effet.

Nuance. Deux mécanismes pour retrouver les 90 secondes : 1. **Un cache de build partagé**, exporté vers le registry — `docker build --cache-to type=registry,ref=.../cache --cache-from type=registry,ref=.../cache`. Le cache devient une image, donc accessible à tous les agents. 2. **Un cache mount BuildKit** (`--mount=type=cache,target=/root/.m2`) combiné à un agent ou un volume persistant, ou un miroir de dépendances interne (Nexus, Artifactory), qui ramène le téléchargement sur le réseau local.

Attention au piège : sans `--cache-from`, un agent éphémère ne bénéficiera **jamais** du cache, quel que soit le soin apporté au Dockerfile.

Exemple.

```
docker build \
  --cache-from type=registry,ref=registry.interne/monapp/api:cache \
  --cache-to type=registry,ref=registry.interne/monapp/api:cache,mode=max \
  -t registry.interne/monapp/api:1.4.2 .
```

Question 9 — Ce que coûte le distroless

Réponse. Deux capacités perdues :

1. `docker exec -it conteneur sh` — il n'y a pas de shell. Impossible d'aller lire un fichier, lancer un `curl` de test, ou inspecter l'environnement en direct.
2. **Les outils de diagnostic** — pas de `ps`, `netstat`, `top`, ni de gestionnaire de paquets pour en installer. Un `jstack` ou un `jcmd` n'est disponible que si l'image distroless choisie embarque un JDK, ce qui n'est pas le cas des variantes minimales.

Compensations habituelles : pour la première, les **conteneurs de débogage éphémères**, qui partagent les namespaces du conteneur cible (`docker debug`, ou en Kubernetes les *ephemeral containers* / `kubectrl debug`) — on apporte les outils sans les avoir dans l'image. Pour la seconde, une **observabilité intégrée** : métriques exposées par l'application (Micrometer, `/actuator`), logs structurés sur `stdout`, traces distribuées. On cesse d'aller chercher l'information à la main, l'application la publie.

Nuance. L'arbitrage est réel. Une équipe sans culture d'observabilité qui passe au distroless se retrouve aveugle au premier incident. Une variante intermédiaire existe : les images distroless `:debug`, qui ajoutent un busybox — pratique, mais qui annule une partie du bénéfice si on les déploie par défaut.

Exemple.

```
docker run --rm gcr.io/distroless/java21-debian12 sh    # échoue : pas de shell
```

Question 10 — Le cache mount de BuildKit

Réponse. Les données sont stockées dans le **cache de BuildKit**, sur le daemon, en dehors de toute couche d'image. Elles n'apparaissent pas dans l'image parce que le montage n'existe que pendant l'exécution de l'instruction `RUN` : au moment de figer la couche, le point de montage est démonté et son contenu n'est pas capturé.

Pourquoi. BuildKit distingue ce qui est *matérialisé dans la couche* de ce qui est *monté temporairement*. Seul le premier entre dans l'image.

Nuance. La différence avec `VOLUME` est totale, et la confusion fréquente : `VOLUME` est une instruction d'**exécution** (elle déclare qu'un dossier du conteneur devra être un volume au `docker run`), le cache mount est une instruction de **build**. L'un concerne les données de vos utilisateurs, l'autre les dépendances de votre compilateur. Ils n'ont ni le même cycle de vie, ni le même contenu, ni le même moment d'existence. À noter aussi : ce cache est local au daemon et disparaît avec `docker builder prune` — il accélère, il ne garantit rien.

Exemple.

```
docker system df                # ligne "Build Cache"
docker builder prune --filter until=168h    # purge les caches de plus d'une semaine
```

Question 11 — « Le multi-stage ne sert à rien pour Angular »

Réponse. Il confond le **résultat** et le **moyen de l'obtenir**. Sans multi-stage, l'image contient tout ce qui a servi à produire le statique : Node (~150 Mo), `npm`, le dossier `node_modules` (300 Mo à 1 Go), le code source TypeScript, la configuration de build et les tests. On passe d'une image finale d'environ **50 Mo** (nginx:alpine + quelques Mo de statique) à **600 Mo à 1,2 Go** — un facteur 15 à 25.

Pourquoi. Le fait que le livrable soit statique ne fait pas disparaître l'outillage utilisé pour le fabriquer : il faut explicitement ne pas l'emporter.

Nuance. Le gain de sécurité est ici encore plus net que pour Java. `node_modules` contient des centaines de paquets tiers, principale source de vulnérabilités signalées par les scanners — alors qu'aucun de ces paquets n'est exécuté en production, puisque seul du JavaScript compilé est servi au navigateur. Sans multi-stage, on hérite d'un rapport de vulnérabilités entièrement hors sujet.

Exemple.

```
docker images --format 'table {{.Repository}}\t{{.Tag}}\t{{.Size}}' | grep web
# web    mono-stage    980MB
# web    multi-stage    52MB
```

Question 12 — `stat /app/dist: no such file or directory`

Réponse. Le `WORKDIR` du stage de build est `/src`, pas `/app`. Le `COPY --from` doit donc pointer sur `/src/dist` :

```
COPY --from=build /src/dist /usr/share/nginx/html
```

Pour diagnostiquer sans deviner, on construit **jusqu'au stage de build** et on l'inspecte :

```
docker build --target build -t debug-build .
docker run --rm -it debug-build sh -c 'pwd && ls -la && ls -la dist'
```

Pourquoi. `--target` arrête la construction au stage nommé et en fait une image utilisable. C'est l'outil de débogage du multi-stage : au lieu de raisonner sur ce que le Dockerfile est censé produire, on regarde.

Nuance. Deux causes voisines produisent le même message et méritent d'être vérifiées dans la foulée : Angular génère depuis la version 17 un sous-dossier supplémentaire (`dist/<nom-du-projet>/browser`), ce qui casse les Dockerfiles écrits pour les versions antérieures ; et un `dist` présent dans `.dockerignore` peut faire échouer un build qui fonctionnait. Le message d'erreur est le même, la cause non — d'où l'intérêt d'aller regarder plutôt que de corriger au jugé.

Exemple.

```
docker run --rm -it debug-build sh -c 'find /src/dist -maxdepth 3 -type d'
```

Question 13 — `RUN mvn test` dans le Dockerfile

Réponse. Dans le **stage de build**, après la compilation et avant le `package` (ou via un stage dédié `AS test`). Un test rouge fait échouer le `docker build`, donc aucune image n'est produite.

```
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml . ; RUN mvn -q dependency:go-offline
COPY src ./src
RUN mvn -q test
RUN mvn -q package -DskipTests
```

Inconvénients. Les tests sont rejoués à chaque build d'image, y compris lorsqu'on reconstruit sans changement de code — le cache de couches limite l'effet, mais le premier build sur un agent neuf paie tout. Surtout, les **rapports de tests sont enfermés dans un stage jeté** : la CI ne peut ni les publier, ni afficher quel test a échoué, ni produire de couverture. On perd le diagnostic au moment précis où on en a besoin. Enfin, les tests d'intégration nécessitant une base de données sont difficiles à faire tourner dans un build (pas de réseau applicatif, pas de services annexes).

Nuance. Le compromis courant en entreprise : la CI exécute les tests **avant** le build d'image, avec publication des rapports et de la couverture, et le Dockerfile ne fait que construire l'artefact. La garantie « pas d'image sans tests verts » est alors assurée par le pipeline, pas par le Dockerfile. Le `RUN mvn test` reste utile quand on veut qu'un `docker build` isolé soit auto-suffisant.

Exemple.

```
docker build --target test -t api:test .    # rejouer uniquement les tests
```

Question 14 — Une couche de 250 Mo contre cinq couches de 280 Mo

Réponse. La seconde. Lors d'une mise à jour du code, seule la petite couche applicative change : les quatre couches stables sont déjà présentes sur les nœuds et dans le registry. On transfère quelques Mo au lieu de 250.

Pourquoi. Le transfert et le stockage sont différentiels par couche. 30 Mo de plus au total est un mauvais calcul sur le **premier** déploiement, et un excellent calcul sur tous les suivants.

Nuance. La réponse s'inverse dans deux situations. D'abord, sur un **premier déploiement** ou sur un nœud neuf : là, on transfère 280 Mo au lieu de 250. Ensuite, quand c'est justement une **couche basse** qui change — montée de version du JRE, correctif de sécurité de l'image de base : toutes les couches suivantes sont invalidées, et le découpage n'apporte alors rien. D'où la règle : découper selon le **rythme de changement réel**, pas par principe.

Exemple.

```
docker pull registry.interne/monapp/api:1.4.3
# Already exists <- les couches stables
# Pull complete <- seule la couche applicative
```